Unit 2: Python Tools for Data Handling: GUI, APIs, Web Data and Databases

Build interactive applications • Fetch data from the web • Scrape web content • Work with databases • Handle and process data

Python powers data!

1. Python GUI Programming Basics

- Introduction to GUI development options
- Building simple interfaces with Tkinter widgets



Tkinter widgets (create the interface)

- Label
- Entry
- Button

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.title("Simple GUI")
label = ttk.Label(root, text="Enter your name:")
entry = ttk.Entry(root)
button = ttk.Button(root, text="Submit")
button pack()
root.mainloop()
```

GUI code (using Tkinter)

2. Working in Jupyter Notebook/IPython

- Interactive data science environment
- Run Python code, visualize results, explore data

jupyter

Run code in a cell

```
In [1]: import pandas as pd
        df = pd.read_csv("data.csv")
        df.head()
```

Out[1]:

	Name	Age	City
0	Alice	25	Delhi
1	Bob	30	Mumbai
2	Charlie	22	Bangalore

View output (data, plots, etc.)

Jupyter Notebook

IPython

3. Internet Programming – APIs

- Network Scripting basics
- Working with REST APIs using the requests library (JSON responses, authentication basics)

REST API

HTTP request (using requests)

```
import requests
url = "https://jsonplaceholder.typicode.com/posts/1"
response = requests.get(url)
data = response.json()
print(data)  # JSON response
```

JSON Response

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt",
  "body": "..."
}
```

Authentication (basic example)

```
headers = {"Authorication": "Bearer <token>"}
response = requests.get(url, headers=headers)
```

Authentication (with headers)

4. Web Scraping Basics

- Extract data from websites using BeautifulSoup



Parse HTML and extract data

BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com/products"
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')
products = soup.find_all('div', class_='product')
for product in products:
    name = product.find('h2').text
    price = product.find('span', class_='price').text
    print(name, price)
```

Scraping code (BeautifulSoup)

Extracted Data

Product Name	$10
Product 2	$20
Product 3	$30
...	...

5. Databases and Persistence

- SQL fundamentals
- Connect Python to databases using sqlite3/SQLAlchemy
- Working with data structures, text and language for data handling

Database connection (using sqlite3 / SQLAlchemy)

SQLite

SQLAlchemy

SQL operations (create, insert, select)

```
import sqlite3
conn = sqlite3.connect('data.db')
cursor = conn.cursor()
cursor.execute("CREATE TABLE users (id INTEGER, name TEXT)")
cursor.execute("INSERT INTO users VALUES (1, 'Alice')")
conn.commit()
cursor.execute("SELECT * FROM users")
print(cursor.fetchall())
conn.close()
```

Users Table

id	name
1	Alice
2	Bob
3	Charlie

6. Working with Data Structures, Text and Language

- Use lists, dictionaries, strings and basic NLP tools for data handling

Lists (store data)

```
data = [1, 2, 3, 4]
print(data[0])
```

Accessing element

Dictionaries (key-value pairs)

```
info = {'name': 'Alice',
        'age': 25}
print(info['name'])
```

Accessing value

Text Processing

```
text = "Python is great!"
words = text.split()
print(words)
```

Tokenizing text

Basic NLP (using NLTK)

```
import nltk
nltk.download('punkt')
tokens = nltk.word_tokenize(text)
```

Language processing

Text · Lists · Dictionaries · NLP

Get Data → Process → Analyze → Build Smarter Apps

Python + GUI + APIs + Web Scraping + Databases + Data Handling = Real-World Solutions

PYTHON TOOLS FOR DATA HANDLING: DATA INTERFACES

1. PYTHON GUI PROGRAMMING BASICS – USER INTERFACES



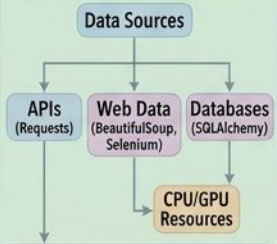




2. API, WEB DATA, AND DATA INTERFACE MODULES



Data Access & Integration Modules



Description	
API Requests	Requests library
Web Scraping	BeautifulSoup
ORM/DB Drivers	SQLAlchemy, psycopg2
Data Interchange	JSON, CSV

```
import requests;
response = requests.get('api/data');
data = response.json();
print(data)
```

Resource Objects

```
import airflow
from airflow import DAG
from airflow.operators.python
import PythonOperator
data = ...
print(data)
```



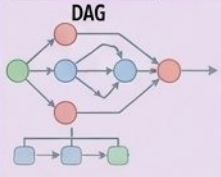
3. Tools & Workflows – Resilient DATA Management



Resilience and Debugging

Tool Method	Description
Data Pipelines	Airflow/Prefect
Distributed Data	Dask/Ray
Resilient Integration	Message Queues, Retry Logic
Monitoring Data	Grafana, logging

Workflow Management (Data flow analysis)



Integration Tools	
Tool	Usage
Apache Airflow	Workflow management
requests	API requests
BeautifulSoup	Web scraping
SQLAlchemy	Database ORM

So, frameworks provide core primitives. Management tools handle e-objects and resources, and a good design integration tools for complex logic and resilience. Monitoring is key!

Scalable tools + efficient management = powerful and reliable data handling!

INTRODUCTION TO GUI DEVELOPMENT & TKINTER BASICS

1. Exploring GUI Development Options & Tools



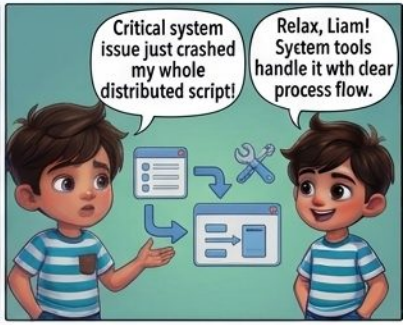
Standard Libraries & Toolkits

Tkinter (Built-in)
Qt (via PyQt/PySide)
Kivy (Cross-platform)
WxPython (Native look)



**tkinter* is the standard, simple entry. *Ot* is powerful but has dependencies.
**Kivy* is mobile-focused.
**wsPython* offers native look.

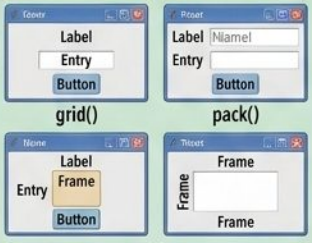
```
1 # Open a minimal window
2 import Tkinter
3 def cab:
4     prise__nanee_tonse;)
5     prisn_poosone()
6
```



2. Building With Tkinter Widgets: Layouts and Elements



Tkinter Widgets and Geometry Managers



Common Widgets and Usage	
Label	(Text display)
Button	(Action trigger)
Entry	(Single-line input)
Text	(Multi-line input)
Frame	(Container)

LEGB
Widget
Widget
Label
Pack side priorities: top, leN... Inore, LEGIR, Pack Frame etc.

```
lbl = Label(root, text='Name');
entry = Entry(root);
btn = Button(root, text='Submit')
```



3. Tkinter Event Handling & Building Simple Apps



Events and Callbacks

Event Type	Description
Button Click	Button widgets
Key Press	Key press
Mouse Motion	Mouse motion vitleets
Menu Select	Menu corrern selects

Event-Driven Programming
(User action flow analysis)

User Action → Event → Callback Function

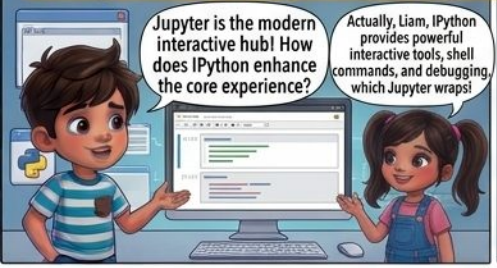
Integration Tools	
Tool	Usage
tkinter.ttk	Styled widgets
requests	API access from GUI
beautifulsoup4	Web content extraction
sqlalchemy	ORM database connection

So, frameworks provide standard primitives. Management tools handle event-objects and widgets, and a good design integration tools for custom logic and responsiveness. Debugging is key!

Scalable widgets + efficient interaction = powerful and reliable GUI handling!

MASTERING JUPYTER NOTEBOOK & IPYTHON FOR DATA SCIENCE

1. Exploring Jupyter Notebooks & IPython Core Features

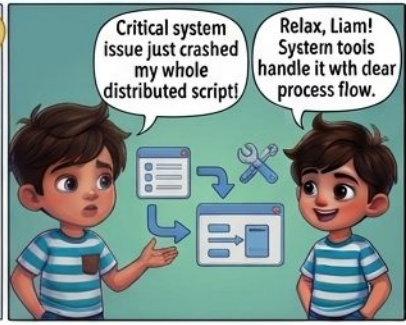


Jupyter Ecosystem & Tools

Jupyter Notebook (Main UI)
IPython Kernel (Core Execution)
Markdown (Documentation)
Code Execution Cells

- Interactive widgets
- Interactive width tools
- Rich display

```
1 !ls
2
3 %%pylab inline
4
5 %%pylab inline
```



2. Interactive Workflow & Data Visualization



IPython Interactive Power & Magics

Data Science Flow

Data Load → Data Process → Analyze → Visualize

%%writefile - an IPython magics are interetate, ootions of interactive and ait hur data ideas.

%pylab inline
Hiis nowe widgets, % pylab inline

%matplotlib inline
Relion ermnte: %matplotlib inline

%timeit - at IPython magics to turn ectiuative widgets.

```
import pandas as pd;
df = pd.read_csv('data.csv');
df.head();
df.plot();
```



3. Advanced Features & Documentation



Cell Types & Metadata

Cell Type	Description
Markdown	Interactive widgets
Code	Code possation tools
Raw	Raw proveded
Metadata	Metadata description

Integration & Publishing
(User event driva flow)

User Action → Event → Book publishing

Integration Tools	
Tool	Usage
nbconvert	Exporting
nbviewer	Viewing
jupyter_book	Book publishing

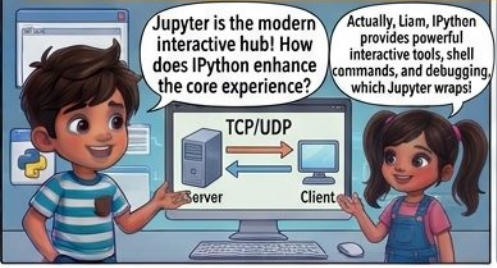
```
$nbconvert myfile.ipynb --to html
```

So, frameworks provide standard primitives. Management tools handle event-objects and widgets, and a good design integration tools for custom logic and responsiveness. Debugging is key!

Scalable widgets + efficient interaction = powerful and reliable GUI handling! 😊

PYTHON NETWORK SCRIPTING BASIC CONCEPTS

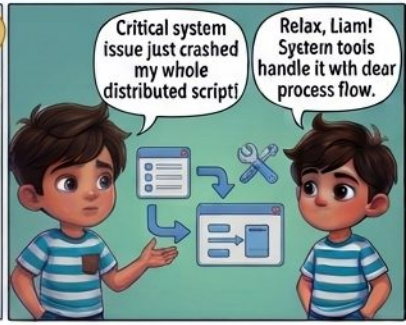
1. Exploring Socket Programming & Network Core Features



Python Network & Scripting Tools

- 'socket' (Low-Level Interface)
- 'http.client' (HTTP Requesting)
- 'json' (Data Parsing)
- 'concurrent.futures' (Concurrency)
- Blocking/non-blocking
- Connection connecting
- Connection types

```
# Create socket
s = socket.socket()
s.bind(('localhost',80))
```



2. Creating Clients & Simple Server Scripts



Python Client-Server Flow & Commands

Network Scripting Flow

Client (Initiate)
↓
Connect
↓
Request
↓
Receive
↓
Parse Data

Common Network Libraries & Tasks

Library	Description
socket	Raw connection
requests	API access
beautifulsoup4	Web content extraction
json	Data encoding/decoding

```
s = socket.socket();
s.connect(('api.data', 80));
s.send(request);
data = s.recv(); print(data.decode())
```



3. Advanced Network Features & Robustness



Data Parsing & Protocol Objects

Protocol	Description
TCP	Connection-oriented
UDP	Connectionless
HTTP	Web protocol
JSON	Data object representation

Concurrency & Robustness
(User event drive flow)

- Multi-threading
- Multi-processing
- Timeout handling
- Retry logic

Multi-process → Task Queue → Network
Worker Process → Result

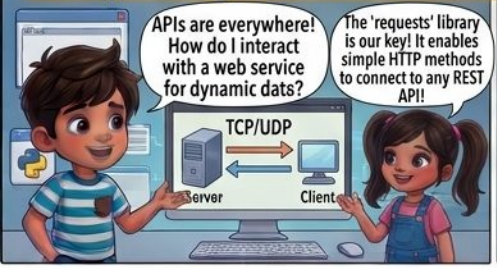
```
import concurrent.futures;
with concurrent.futures.ProcessPoolExecutor() as executor:
    executor.submit(send_data, ip_list)
```

So, frameworks provide core primitives. Management tools handle event-objects and resources, and a good design integration tools for custom logic and scalability. Debugging is key!

Scalable widgets + efficient interaction = powerful and reliable GUI handling!

PYTHON REST APIs – mastering the requests library

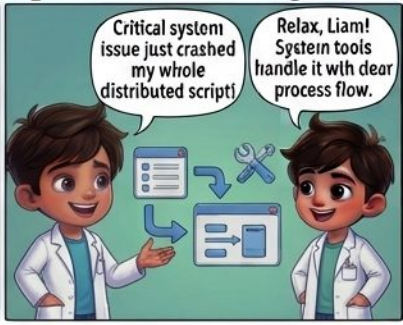
1. Exploring REST API Basics & The Requests Core



Python requests Library & Network Concepts

- 'mport requests' (Core module)
- HTTP Methods (GET, POST, PUT)
- JSON Data Parsing ('.json()')
- Blocking/non-blocking
- Connection connecting
- Connection types

```
# Create socket
s = sosket.socket()
s.bind(('localhast',80))
```



2. Creating Clients & Simple API Scripts



Python Clienon API Libraries & Tasks

HTTP Request Flow

GET Request → API → Server → Response (e.g. JSON) → Parse Data

Common API Libraries & Tasks

Library	Description
requests	Raw HTTP
http.client	Raw network connection
beautifulsoup4	Web content extraction
json	Data encoding/decoding

```
response = requests.get('https://api.data.com');
data = response.json();
print(data)
```



3. Advanced API Features & Authenication



API authentication & Authorization

Method	Description
Basic Auth	Username:Password
API Keys	Header/Query param
Token objects	e.g., Bearer token
Session tokens	

Integration Tools

Tool		Usage
requests.auth	auth	Authentication methods
oauthlib	oauthlib	Advanced authorization
json	json	Token object handling
retry	retry	Advanced retry logic

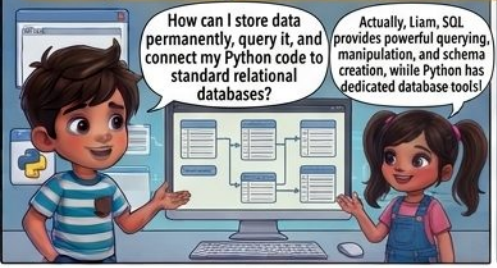
```
inport requests
auth = requests.auth.HTTPBasicAuth('user', 'pass')
requests.get(url, auth=auth)
```

So, frameworks provide core primitives. Management tools handle event-objects and resources, and a good design integration tools for custom logic and scalability. Debugging is key!

Flexible clients + efficient handling = powerful and reliable API interaction!

MASTERING PYTHON DATABASES & PERSISTENCE

1. Exploring SQL Fundamentals & Core Concepts



SQL Basics & Key Concepts

- Data Types
- Table Schema (CREATE)
- Data Insertion (INSERT)
- Queries (SELECT)



- *CREATE* is a schema. Data inside is standard.
- *SELECT* is for querying. Schema logic is core.

```
# Open a minimal SQL
# query
> = sqlite.socket()
s.bindd('sqlite3')
```



2. Interactive Workflow & SQL Operations



SQL Operations Flow & Commands

Data Access & Connection Flow

Open Connection → Execute SQL → Process Result → Close Connection

SQL Operators	Integration Tools	
	Tool	Usage
SELECT	sqlite3	Built-in database
FROM	requests	API access from GUI
WHERE	sqlalchemy	Powerful ORM/DB connecting
JOIN	psycopg2	DB-API driver for PostgreSQL

```
import sqlite3
conn = sqlite3.connect('my.db');
cur = conn.cursor();
cur.execute('SELECT * FROM users'); cur.fetchall()
```



3. Unpacking Advanced Database Topics & Workflows



Advanced Database Design & ORM Concepts

Topic	Description
Normalization	Data structure optimization
Normalization	Data structure optimization
Indices	Query performance optimization
ORM/Table Classes	Abstraction for data handling

Workflow & Integration Tools

Workflow Management (Data flow analysis)

SQLAlchemy ORM → Python Classes → SQL → Query Results



Integration Tools	
sqlite3	Built-in standard
sqlalchemy	Powerful and flexible connecting
json	Token object handling
psycopg2	DB-API driver for PostgreSQL

So, frameworks provide core primitives. Management tools handle event-objects and resources, and a good design integration tools for custom logic and scalability. Debugging is key!

Scalable widgets + efficient interaction = powerful and reliable GUI handling!

MASTERING DATA STRUCTURES, TEXT, & LANGUAGE

1. Exploring Data Structures & Language Concepts



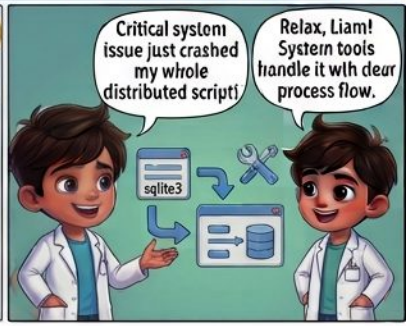
Data Basics & Keyry Contepts

- Data Types
- Lists & Dicts (Storage)
- Regex & Parsing (Cleaning)
- NLP & Language Analysis

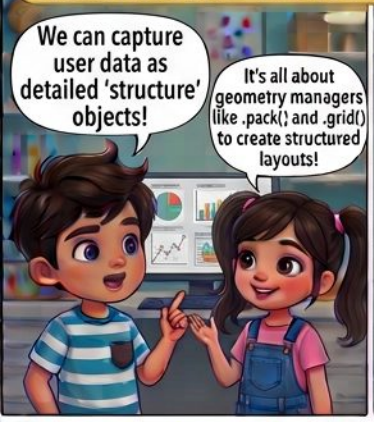
Key terms for content, while cls standardd is powerful, libraries liks Pandas are dedicated data handling tools.

- *List* is a sequence. *Dict* is a map.
- Regex is for cleaning. Schema logic is core.

```
# Data a minimel list
i+teona = [list()
df = rcede.zax(['sbees''])
> print('list>st)
```



2. Interactive Workflow & Text Operations



Python Structures Flow & Language API Commands

Data Access & Connection Flow

Input Data → Structure (e.g., DataFrame) → Process (e.g., Filter) → Language API

Standard Python built-ins	
Len()	len(), dir() len(), dir()
dir()	Tix operators: get(), split()
Process	Library operators: get(), split()
Result	Library operators: get(), split()

```
import pandas as pd;
df = pd.read_csv('text_data.csv');
cleaned = df['text'].str.replace('^','').split();
print(cleaned)
```



3. Unpacking Advanced Language Topics & Workflows



Advanced Language Processing & Structure Concepts

Cell Type	Description
Regex Parsing	Advanced Cleaning Techniques
Tokenization	Tokenization
POS Tagging	Chunking & POS Tagging
Text Classifiers	Structure for scalability

Workflow Management
(Language data flow analysis)

Data DAG



Integration Tools	
Tool	Usage
pandas	Powerful handling
nltk	Language analysis
requests	API access from GUI
json	Token object handling

So, frameworks provide standard primitives. Management tools handle event-objects and resources, and a good design integration tools for custom logic and scalability. Debugging is key!

Flexible structures + efficient processing = powerful and reliable data handling! 😊